

SQL QUERIES :

query 1 to answer 1st how much is each member borrowing?

```
query1 = """
```

```
SELECT
```

```
    members.first_name || ' ' || members.last_name AS Name,
```

```
    COUNT(checkouts.book_id) AS Books_borrowed
```

```
FROM members
```

```
LEFT JOIN checkouts
```

```
    ON members.member_id = checkouts.member_id
```

```
GROUP BY members.member_id, members.first_name || ' ' || members.last_name
```

```
ORDER BY Books_borrowed DESC;
```

```
""" result1 = pd.read_sql(query1, conn)
```

```
result1      80 rows × 2 columns
```

Name	Books_borrowed	
0	Aya Wahba	25
1	Sherif Saleh	21
2	Ziad Saleh	19
3	Nour Nabil	18
4	Mostafa Fouad	18
...	...	...
75	Layla Fouad	0
76	Fares Sabry	0
77	Amir Wahba	0
78	Bassel Adel	0
79	Habiba Osman	0

The letter chosen "A"

query 2 to answer 2nd which books match a chosen author pattern?

```
query2 = """
```

```
SELECT *
```

```
FROM books
```

```
WHERE author LIKE 'A%';
```

```
"""
```

```
result2 = pd.read_sql(query2, conn)
```

```
result2
```

	book_id	title	author
0	501	The Silver Kite	Amina Darwish
1	502	Desert Compass	Amina Darwish
2	503	The Lantern Maker	Adel Roushdy
3	504	Rooftop Astronomers	Adel Roushdy
4	505	Letters to the Nile	Aya Hafez
5	506	The Paper Boat Club	Aya Hafez

```
# query 3 to answer 3rd what are the most popular books?
```

```
query3 = """
```

```
SELECT
```

```
    books.title AS Title,
```

```
    COUNT(checkouts.book_id) AS checkouts_count
```

```
FROM books
```

```
JOIN checkouts
```

```
    ON books.book_id = checkouts.book_id
```

```
GROUP BY books.book_id, books.title
```

```
ORDER BY checkouts_count DESC
```

```
LIMIT 5;
```

```
"""
```

```
result3 = pd.read_sql(query3, conn)
```

```
result3
```

	Title	checkouts_count
0	The Silver Kite	57
1	Fossils and Fireflies	55
2	Circuits for Beginners	46
3	Kites Over Cairo	38
4	Storms and Sailboats	25

```

# query 4 to answer 4th who are the most active readers?
query4 = """
SELECT
    members.first_name || ' ' || members.last_name AS Name,
    COUNT(checkouts.book_id) AS Books_borrowed
FROM members
JOIN checkouts
    ON members.member_id = checkouts.member_id
GROUP BY members.member_id, members.first_name || ' ' || members.last_name
ORDER BY Books_borrowed DESC
LIMIT 10;
"""

result4 = pd.read_sql(query4, conn)
result4

```

	Name	Books_borrowed
0	Aya Wahba	25
1	Sherif Saleh	21
2	Ziad Saleh	19
3	Nour Nabil	18
4	Mostafa Fouad	18
5	Ahmed Shafik	17
6	Youssef Hegazy	17
7	Adam Fahmy	17
8	Reem Osman	16
9	Sara Rashad	16

Chosen neighborhood "Maadi"

query 5 to answer What does a neighborhood's activity look like further back in time?

```
query5_neighborhoods = """
SELECT DISTINCT neighborhood
FROM members;
"""
neighborhoods = pd.read_sql(query5_neighborhoods, conn)
print ("Maadi")
query5 = """
SELECT
    checkouts.*
FROM checkouts
JOIN members
    ON checkouts.member_id = members.member_id
WHERE members.neighborhood = 'Maadi'
ORDER BY checkouts.checkout_date DESC
LIMIT -1 OFFSET 10;
"""
result5 = pd.read_sql(query5, conn)
result5
```

	checkout_id	member_id	book_id	checkout_date	return_date
0	9103	1003	513	2025-09-04	2025-09-25
1	9081	1017	502	2025-08-25	2025-09-17
2	9001	1008	501	2025-08-23	2025-08-28
3	9050	1003	521	2025-08-21	None
4	9085	1018	525	2025-08-19	2025-09-18
...	...	...	...	...	...
74	9046	1022	513	2024-02-22	None
75	9013	1002	501	2024-02-16	2024-02-29
76	9043	1005	505	2024-01-26	2024-02-04
77	9067	1008	507	2024-01-05	2024-01-29
78	9037	1008	509	2024-01-02	2024-01-22

79 rows × 5 columns

Reasons for choosing each query :

Reasons for choosing query 1 :

I used a `LEFT JOIN` to connect the `members` table with the `checkouts` table using the member ID. This allows me to include all members, even those who have not borrowed any books. I used `COUNT()` to calculate the number of books borrowed by each member, `GROUP BY` to calculate the count separately for each member, and `ORDER BY DESC` to display members from the highest number of borrowed books to the lowest.

Reasons for choosing query 2 :

I used the `books` table because it contains the book and author information. The `LIKE 'A%'` condition was used to find books whose author's name starts with the letter "A". The `%` wildcard allows any characters to appear after the letter A. We used `SELECT *` to display all the information about the matching books.

Reasons for choosing query 3 :

I joined the `books` table with the `checkouts` table using `book_id` to find how many times each book was borrowed. I used `COUNT()` to calculate the number of checkouts for each book and `GROUP BY` to calculate the count separately for each book. The results were sorted in descending order using `ORDER BY` so that the most borrowed books appeared first. Finally, I used `LIMIT 5` to return only the five most popular books.

Reasons for choosing query 4 :

I joined the `members` table with the `checkouts` table using `member_id` to determine how many books each member has borrowed. I used `COUNT()` to calculate the number of borrowed books for each member and `GROUP BY` to calculate the count separately for each member. The results were sorted in descending order so that the members with the highest number of borrowed books appeared first. Finally, I used `LIMIT 10` to identify the ten most active readers.

Reasons for choosing query 5 :

I first retrieved the distinct neighborhoods from the `members` table to confirm that the chosen neighborhood, Maadi, exists in the database. Then, I joined the `checkouts` table with the `members` table using `member_id` so that I could filter the checkout activity by neighborhood. I used the `WHERE` condition to select members from Maadi and sorted the checkouts by date in descending order. Finally, `OFFSET 10` was used to skip the ten most recent checkouts, allowing me to examine activity further back in time.

Reflection: Web Scraping, API, and Database

In this project, I worked with data from three different sources: an HTML web page, JSON data collected through an API, and a database. Each source provided the data in a different format. I used web scraping to collect information from the HTML page, the API to get structured JSON data, and SQL to work with the database.

Comparing these sources was important because I noticed that the data was not always exactly the same. For example, the number of checkouts in the database was different from the information I found on the HTML page. This showed me that data from different sources may contain differences and should be checked carefully.

After collecting the data from the different sources, I combined it into one dataset and then started the cleaning process. During this process, I checked for missing values, duplicates, and inconsistent information. Overall, working with HTML, JSON, and database data helped me understand how different data sources can affect the final combined dataset and why comparing them was important for this project.